虚拟化技术综述

从软件模拟到硬件辅助与嵌套虚拟化

《高级操作系统教程》期中调研报告

王靳

中国科学院大学

学号：2026E8013282153

2026 年 11 月 4 日

摘要

虚拟化试图让客户机获得整台机器的使用接口，却始终把对真实资源的最终控制权保留给虚拟化管理层。本文围绕这一核心问题，系统综述从软件模拟到硬件辅助再到嵌套虚拟化的实现路径，范围限定于系统级整机虚拟化。全文以 CPU、内存、中断、设备四维为统一框架，逐维回答资源与状态由谁控制、关键路径如何执行、何时介入以及代价何在。软件一侧以 QEMU 的 TCG 全系统模拟为例，展示翻译执行、软件地址转换、事件状态机与设备模型如何在一个进程内构造出整台机器；硬件一侧以 Armv8.6-A 与 Linux KVM 为例，展示异常级、两阶段地址翻译、GIC 与 SMMU 如何把高频路径的软件介入下放给处理器与系统结构，管理策略与设备语义仍留给软件。本文借 Turtles 与 HyperTurtle 梳理嵌套虚拟化：前者通过合成 VMCS 与多维分页把多层语义组合到单层硬件支持上，后者以受控的 eBPF hyperupcall 把 L1 定义的部分处理逻辑下放到 L0 就地执行、并通过共享状态协议和回退路径协调两层管理。综述判断，虚拟化演进并非软件被硬件简单替换，而是不断调整执行、状态与策略的放置位置；能否在减少跨层往返的同时维持资源控制、状态一致性与上层管理权，才是方案之间真正可比的问题。

关键词：虚拟化；虚拟机监视器；硬件辅助虚拟化；嵌套虚拟化；QEMU；KVM

目录

0　引言：虚拟化的形式要求与工程维度　3

0.1　背景与范围 3

0.2　Popek–Goldberg 定义与充分条件 3

0.3　从形式模型到四维工程框架 4

1　软件虚拟化：QEMU 的全系统实现　5

1.1　CPU 虚拟化：RISC-V 到 TCG IR 再到 x86-64 5

1.2　内存虚拟化：软件 MMU、软件 TLB 与物理地址空间 7

1.3　中断虚拟化：从设备事件到 guest 异常入口 8

1.4　设备虚拟化：MMIO、软件 DMA 与后端服务 9

1.5　能力与成本 10

2　硬件辅助虚拟化：Arm 架构与 KVM　11

2.1　CPU 虚拟化：EL2、选择性陷入与 vCPU 上下文 11

2.2　内存虚拟化：VA 到 IPA 到 PA 的两阶段翻译 12

2.3　中断虚拟化：GICv3 的虚拟接口与 GICv4 的直接注入 13

2.4　设备虚拟化：SMMUv3、DMA 隔离与设备直通 15

2.5　协作小结：硬件负责机制，软件负责管理与组合 16

3　嵌套虚拟化：从 Turtles 到 HyperTurtle　16

3.1　问题建立：当 hypervisor 自己也在虚拟机里 17

3.2　Turtles：把多层语义组合到单层硬件支持上 17

3.3　从” 能嵌套” 到” 少往返”：性能问题的重新定位 20

3.4　HyperTurtle：把 L1 的部分处理逻辑下放到 L0 21

3.5　实验证据评议 23

3.6　局限与可检验的后续问题 24

结语　25

参考文献　27

0　引言：虚拟化的形式要求与工程维度

虚拟化试图解决一个看似矛盾的问题：让一台客户机（guest）获得整台机器的使用接口，却始终不把对真实机器资源的最终控制权交给它 [1]。本篇综述围绕这一中心问题，沿着一条从纯软件到硬件辅助再到嵌套组合的路径展开，考察一台虚拟机器如何先在一个普通进程里被完整构造出来，哪些高频操作可以改由 CPU、内存管理单元、中断控制器和 IOMMU 承担，以及当把 hypervisor 本身也当作客户运行时，资源映射、事件归属和管理策略如何跨层组合、又如何减少层间往返。全文以 CPU、内存、中断、设备四个维度作为统一的分析框架，对每一维度都回答同一组问题：资源与状态由谁控制、关键路径怎样执行、何时需要虚拟化管理层介入、介入的代价何在。第 1 节以 QEMU 的全系统模拟说明软件实现，第 2 节以 Armv8.6-A 与 Linux KVM 说明硬件辅助，第 3 节以 Turtles 与 HyperTurtle 说明嵌套虚拟化，各节在同一四维结构下逐层对照。

0.1　背景与范围

对虚拟化最直接的需求来自多系统共存: 在同一批物理资源上同时运行互不信任、甚至互不兼容的软件栈，需要把资源复用、故障隔离和接口兼容统一在一个机制里。隔离要求一个程序无法触及未分配给它的资源，兼容要求客户操作系统及其应用程序无需修改即可运行，复用则要求多个客户共享底层硬件时仍然各得其所。为此引入的管理层称作虚拟化管理器（hypervisor）或虚拟机监视器（VMM），它在一个真实机器上构造出若干台虚拟机（virtual machine），每台虚拟机对外呈现一套独立的指令集、寄存器、地址空间、中断与设备接口；客户看到的执行单元是虚拟 CPU（vCPU），其物理载体是真实处理器上受 VMM 调度的执行上下文。

本节讨论的范围限定在系统级虚拟机, 即能够承载未修改操作系统的整机虚拟化, 不涉及进程级的语言虚拟机。需要先澄清一个术语上的容易误解: 软件模拟与硬件辅助并不是互斥的产品分类, 而是实现同一目标的两条路径, 且常常并存于一个系统之中。QEMU 既可以独立提供完整的机器模型，以动态翻译的方式执行客户代码，也可以与 KVM 协作，把 CPU 执行路径交给硬件辅助机制，继续承担内存后备、设备模型与整机编排等职责 [2–4]。因此本综述把软件与硬件两部分并置，意在比较同一维度上两种机制的取舍，而不是把二者对立起来。

0.2　Popek–Goldberg 定义与充分条件

对“什么是可虚拟化的机器”这一问题的形式化回答，来自 Popek 与 Goldberg 1974 年的论文。他们先用三条性质给出 VMM 的定义，再在简化机器模型上证明一个可判定的充分条件 [1]。这三条性质是等价性（equivalence）、资源控制（resource control）与效率（efficiency），属于 VMM 的定义本身，而不是三个可以分别取舍的定理。等价性要求任何程序在 VMM 之下的执行效果与它直接在原机器上运行不可区分，但明确允许两类例外：因可用资源数量不同造成的差异，以及因时间依赖（timing dependencies）造成的差异；前者是为了容纳 VMM 分配不同大小内存的情形，后者则是因为中间多了一层软件、且同一硬件上还有其他虚拟机并发存在。资源控制要求 VMM 对内存、外设等资源握有完全控制权，一方面使运行于其下的程序不可能访问任何未显式分配的资源，另一

方面使 VMM 在规定条件下能够重新取得对已分配资源的控制。效率要求则最严：统计上占主导地位的那一子集客户指令必须直接由真实处理器执行，VMM 完全不介入；正是这一条把传统的解释器和纯软件模拟器排除在虚拟机的定义之外。

充分条件的关键在于区分敏感指令（sensitive instruction）与特权指令（privileged instruction）这两个不同概念。特权指令是机器的固有属性：在用户态执行会陷入、在管态执行则不陷入。敏感指令则分两类：控制敏感（control sensitive）指令试图在不过内存陷入序列的情况下改变可用资源数量或处理器模式，行为敏感（behavior sensitive）指令的执行效果依赖于它在真实内存中的位置或当前模式，行为敏感指令中的位置敏感例子是取得物理地址，模式敏感例子是访问上一执行模式的指令空间。敏感与特权并非同义词，一个机器即使具备管态与用户态两种模式，也未必满足“敏感集合包含于特权集合”；若存在某条敏感指令在用户态不陷入，仅仅把操作系统降权运行就无法保证等价性与资源控制同时成立。论文的定理 1 给出的是充分条件而非必要条件：在满足其全部机器模型假设的第三代式机器上，只要敏感指令集合是特权指令集合的子集（sensitive $\subseteq$ privileged），即可构造出满足三条性质的 VMM。trap-and-emulate 是对这一条件的直观解释——敏感指令在用户态自然陷入，由 VMM 代为模拟其语义——但它描述的是该条件下一种可行的实现方式，并不能外推为所有现代实现的必要条件。

这一形式结果有两个边界必须说清。其一，定理依赖的机器模型把状态记作可执行存储、处理器模式、程序计数器与 relocation-bounds 寄存器四元组，并显式假设 I/O 指令与中断不存在，尽管论文指出它们可作为扩展加入 [1, §2]。因此该模型覆盖的是处理器状态与一种简化的重定位内存，没有把完整中断与设备行为纳入形式证明。其二，论文第 7 节已经讨论递归虚拟化，即 VMM 能否在其自身上再运行一份 VMM，并给出定理 2：一台第三代式机器若满足两个前提即可递归虚拟化——它本身可虚拟化，且能够为它构造出一个无时间依赖的 VMM。这里的递归条件不承诺固定不变的低开销，每增加一层 VMM 都要消耗资源，实际实现还可能增加调度、状态切换与陷入处理的成本；第 3 节将回到这一点，讨论其工程实现与代价，而不是宣称定理在第二层失效。

0.3　从形式模型到四维工程框架

Popek–Goldberg 的形式模型为“可虚拟化”提供了可检验的判据，但它的抽象范围与现代整机虚拟化所面对的对象并不重合。它把内存抽象成一条 relocation-bounds 寄存器驱动的简单重定位，把 I/O 指令与中断整个排除在外；而真实系统里的内存虚拟化涉及多级页表与访问权限，中断虚拟化涉及路由、pending 状态与投递时机，设备虚拟化则要处理 MMIO、DMA 与设备状态。要把理论判据落到工程实现上，就需要把分析对象补全。表 1 给出本文采用的四个维度，它用于分解整机虚拟化所需的机制，是后续各节的共同分析框架，而非 Popek–Goldberg 原文提出的分类；中断与设备在物理上虽有交叉，但本文把设备维度留给功能与数据通路，把中断维度留给异步控制流，分别讨论。

最后还需划定一个术语边界。本文把 QEMU 的跨指令集全系统模拟纳入广义的“软件虚拟化”，但必须指出它与 Popek–Goldberg 意义上的 efficiency 不重合：efficiency 要求统计上占主导的客户指令由真实处理器直接执行，而 QEMU 将 RISC-V 指令翻译成中间表示再生成 x86-64 代码执行，二者并非同一指令在真实处理器上的直接执行，因此并不满足这条严格定义 [1, 3]。翻译块（TB）缓存降低的是反复翻译与解码的模拟开销，它提

表 1　四维分析框架：各维度的对象、控制目标与共同问题

维度	客户得到什么	VMM 必须控制什么	各章共同回答的问题
CPU	指令集、寄存器、特权状态与 vCPU	执行上下文、敏感操作、运行与暂停	指令由谁执行？何时需要 VMM 介入？
内存	自己的物理地址空间与页表	真实内存分配、地址映射与访问权限	客户认为的“物理地址”最终指向哪里？
中断	定时器及设备产生的异步事件	中断路由、pending 状态、目标 vCPU 与投递时机	事件如何送到正确的客户，并遵守其屏蔽与优先级规则？
设备	可发现、可编程的设备接口	MMIO、DMA、设备状态与后端资源	寄存器访问、数据传输和完成通知分别经过谁？

高的是模拟效率，不能把它偷换成“直接执行”。这一区分并非贬低软件路径的价值：软件路径不依赖宿主提供目标架构的虚拟化扩展；在模拟器具有对应 CPU、设备模型及可用宿主后端的前提下，它能在异构平台上运行整套 guest 软件。下一节就从这条软件路径出发，看一台虚拟机器如何在 QEMU 进程内部被逐维度地构造出来。

1　软件虚拟化：QEMU 的全系统实现

本章考察纯软件方式构造整台机器的路径：以 `qemu-system-riscv64` 的 TCG 全系统模拟为对象，guest 为 RISC-V，host 为 x86-64，机器采用 `virt` 平台的 CLINT 与 PLIC 配置，不涉及 RISC-V H 扩展或 AIA[3]。这一配置下，宿主处理器不执行任何 guest 指令，guest 的 CPU、内存、中断控制器与外围设备全部由运行在宿主用户态的一个 QEMU 进程以软件数据结构和方法调用来实现。QEMU 用翻译代码和软件例程实现 guest 的状态转移。CPU 指令翻译只是其中一环，地址空间、中断和设备必须共同提供一致的整机接口。

1.1　CPU 虚拟化：RISC-V 到 TCG IR 再到 x86-64

guest CPU 的机器状态首先表现为 QEMU 进程内的一组 C 结构。RISC-V 的通用寄存器、PC、CSR 与特权级都保存在 `CPURISCVState` 中，guest 的当前特权级只是该结构里的一个字段，它与承载 QEMU 进程的宿主线程实际运行在哪个特权级毫无对应关系：guest 的 M 态操作由生成代码或 helper 实现，而不会因此把 QEMU 线程提升到宿主内核态。QEMU 需要宿主服务时仍可通过系统调用进入宿主内核，这与 guest 的特权级变化无关。这是软件虚拟化与 P&G 直接执行路径最根本的分野，guest 指令不会被宿主硬件执行，因此跨 ISA 的 QEMU 模拟并不满足 P&G 关于统计上占主导的 guest 指令由真实处理器直接执行的效率要求 [1, 3]。

执行的入口是翻译前端。QEMU 把一条 RISC-V 指令先解码，再表达为与目标无关的中间表示 TCG IR，最后才由宿主后端发射 x86-64 代码。这个三级路径（图 1）是 QEMU 可

移植性的来源：前端只负责把某种 guest ISA 的语义翻译成少量、正交的 TCG 操作，宿主后端只负责把 TCG 操作翻译成某一种宿主指令，两者之间用 IR 解耦，新增一个 guest 或 host 都无需改写另一端。TCG 操作包括临时变量的移动、整数算术（如 `add`、`and`）、分支、访存以及 `call` 等[1]。以 `add rd,rs1,rs2` 为例，其译码项落入 `target/riscv/insn32.decode` 的 R 型格式，转译函数 `trans_add` 最终调用 `tcg_gen_add_tl` 发出一次目标宽度加法[2]；该例的算术核心可表达为一条加法 IR，但宿主指令数还取决于操作数装入、状态写回、常量折叠和具体指令选择，不能从 guest 指令数直接推算。访存与 CSR 访问则不同。`lw rd,imm(rs1)` 需要先算出有效地址，再把该 guest 虚拟地址交给软件 MMU 完成地址翻译与权限检查，命中 RAM 时通过缓存的宿主偏移直接取数，未命中则转入 MMU helper，因此会产生地址加法、访存操作以及潜在的一次 helper 调用。CSR 访问（如 `csrr`）涉及权限检查及可能影响后续执行的控制状态，通常由相应 C 语言 helper 完成，而不是内联成算术。这些例子展示了普通 IR、带地址转换语义的访存操作和 helper 的分工。helper 并不限于特权指令，部分普通计算也可调用它；是否内联由具体目标实现决定。

翻译产物组织成 Translation Block（TB），即在一组 CPU 状态假设下生成的可独立查找、链接和失效的代码单元。TB 不等同于编译器定义中的严格单基本块，内部可以出现标号与控制流。主执行循环 `cpu_exec_loop` 按当前 PC 与翻译相关 CPU 状态查找 TB，未命中才调用 `tb_gen_code` 现场翻译，命中则直接执行已缓存的宿主代码[3]。TB 的有效性不只用 PC 作键：TB 记录了自己成立时假设不变的 CPU 状态（例如特权级、若干段基址），状态变化后必须重新查找匹配当前假设的 TB，必要时才生成新的代码。发射之前，TCG 还要对整块 IR 做优化、可达性分析与活跃性分析，再为临时变量分配宿主寄存器，最后由后端生成机器码——这些步骤在翻译时付出；运行时仍要执行转换后的指令、状态访问与 helper，缓存主要摊薄重复翻译的成本 [3]。为了减少每次回到调度循环的往返，QEMU 提供直接块链接：`goto_tb`/`exit_tb` 机制在首次跳转后把跳转槽修补成目标 TB 的宿主地址，之后直接跳转；`lookup_and_goto_ptr` 则用 helper 查表、再 `goto_ptr` 分支 [3]。其中，`goto_tb` 直接链接要求目标在翻译时可确定，并满足同页等映射安全约束；运行时查找目标的间接跳转不应与这种直接链接混为一谈。

helper 是表达 guest 复杂语义的手段，而不是 TB 的边界标记：一个 TB 内部可以多次调用 helper，helper 返回后继续执行后续宿主代码。guest 异常同样存在，只是由软件实现。以本章 RISC-V 全系统模式下的访存异常为例，MMU helper 判定翻译或权限失败，设置 guest 异常信息，并通过 `cpu_loop_exit_restore()` 等路径恢复对应 guest 指令的状态、退出生成代码；目标异常处理逻辑再按架构规则更新 CSR、特权状态和 PC。这里不能把其他 ISA 的除法异常，或 user-mode 的宿主信号处理，直接套入本章路径 [3]。

图 1 把上述过程分成译时与运行时两侧：译时是”RISC-V 解码 →TCG IR→ 优化与寄存器分配 →x86-64 发射” 的流水线，产物放入 TB 缓存；运行时则以 PC 与 CPU 状态查缓存，命中直接执行、未命中回译时，两条路径由块链接与调度循环衔接。需要强调的是，示意中的宿主代码代表语义等价实现，而不是固定的逐指令替换模板。

[1] QEMU: `include/tcg/tcg-opc.h`。

[2] QEMU: `target/riscv/tcg/insn_trans/trans_rvi.c.inc`, `trans_add()`。

[3] QEMU: `accel/tcg/cpu-exec.c`, `cpu_exec_loop()`。

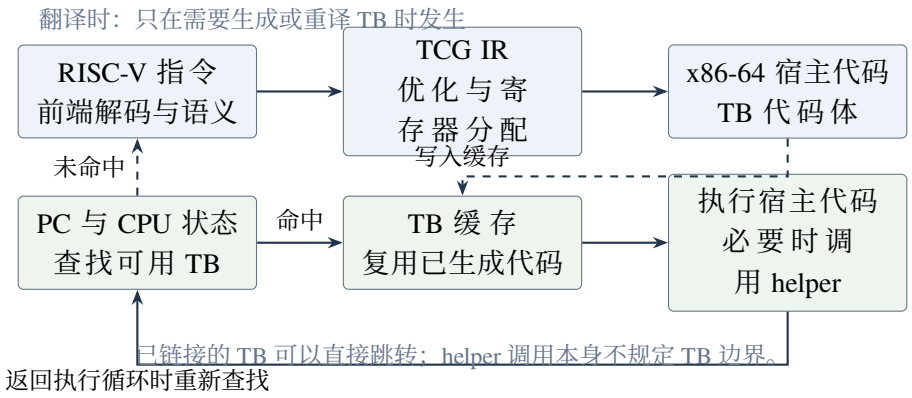


图 1　QEMU 的译时与运行时路径。上层将 guest 语义转换成宿主代码，下层复用代码；图为依据源码和开发文档整理的机制示意，并非实测 trace。[3]

1.2　内存虚拟化：软件 MMU、软件 TLB 与物理地址空间

guest 的每一次访存都必须经过一段软件地址翻译，同时保留 guest 自己页表与权限的语义。地址关系是三层：guest 虚拟地址经 guest 页表得到 guest 物理地址，再落到 QEMU 进程中的宿主虚拟地址；最后由宿主操作系统与宿主 MMU 完成宿主虚拟地址到宿主物理地址的映射——这最后一段不是 QEMU 模拟的 guest 页表阶段，QEMU 并不干预它。

guest 操作系统负责建立它自己的页表，QEMU 的 RISC-V MMU 实现则在每次地址翻译时读取这些页表、逐级检查权限与访问类型，得到 guest 物理地址，或在违反规则时生成一条 guest 可见的访存异常；对 RISC-V 而言，访问类型还须与 PMP 规则再做一次物理内存保护检查，二者都通过才得到最终可用的 guest 物理地址[4]。为把这条慢路径从每个访存的热循环里剥离，QEMU 用软件 TLB 缓存翻译结果：TCG 生成的访存代码先检查缓存，RAM 命中时直接以缓存的宿主地址偏移完成读写，未命中才进入 MMU helper 补表[5]。这里必须区分两种” 未命中”：软件 TLB miss 通常只是缓存里没有这一页，helper 重走 guest 页表后补上即可继续；而 guest 页表本身不合法时产生的是 guest page fault，会被 `raise_mmu_exception` 转成 guest 可见异常。前者是 QEMU 内部的性能事件，后者是模拟出来的架构事件，二者不能混同。

翻译结果之上是物理地址空间的建模。QEMU 用无环的 `MemoryRegion` 图与 `AddressSpace` 对象描述整台机器的地址布局：RAM 区域有宿主内存后备，读写直达宿主缓冲区；MMIO 区域没有宿主内存，每次访问都会转交对应设备模型处理；此外还有 ROM、ROM 设备、别名与容器等类型，用于表达只读固件、可动态改写的窗口等语义[6]。RAM 与 MMIO 在 TLB 填充阶段就被区分：前者缓存了”guest 地址到宿主内存的偏移”，后者设置 MMIO 等慢路径标志，并不提供可直接读写的 RAM 后备地址，使访存转向设备回调 [3]。这种分流既决定了性能（RAM 快、MMIO 慢），也把内存虚拟化自然接到设备虚拟化上。

两类缓存解决的是两类不同的失效问题：地址翻译变化要求维护软件 TLB，代码内容变化要求失效对应的 TB。软件 TLB 根据 guest 虚拟页及 MMU 上下文选择表项；TB

[4]QEMU: `target/riscv/tcg/cpu_helper.c`, `get_physical_address()`。

[5]QEMU: `accel/tcg/cputlb.c`, `tlb_set_page_full()`; `target/riscv/tcg/cpu_helper.c` 中的地址翻译。

[6]QEMU: `docs/devel/memory.rst`。

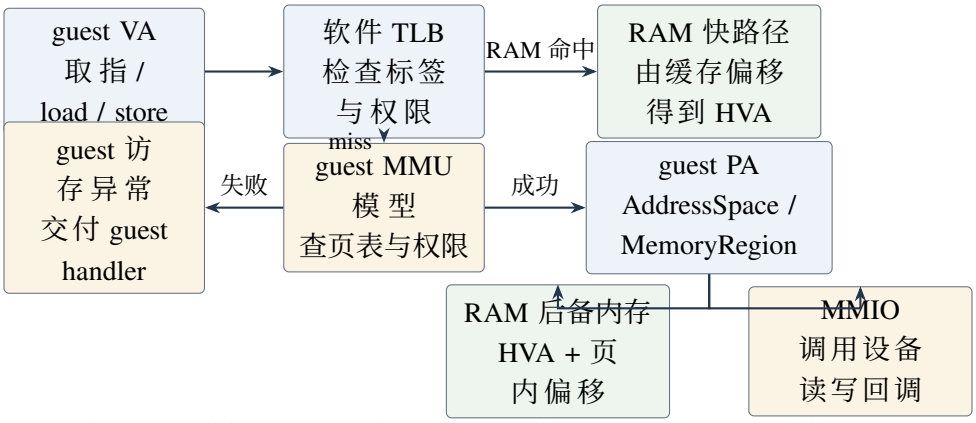


图 2　软件 MMU 的访存路径。软件 TLB miss 触发补充转换，不必然产生 guest 页故障；MMIO 和 RAM 共享 guest 物理地址空间，但执行路径不同。[3]

的物理代码页关联则用于识别被修改的代码并维护失效关系，执行入口另有 PC 相关的快速查找。不能把这些结构统称为物理地址索引的缓存。系统模拟下，软件 MMU 的写入追踪及代码页关联用于失效受影响的 TB，并撤销相应直接链接[7][3]。

图 2 给出一次 load 的分支全景：RAM 快路径命中时使用缓存地址，miss 时进入 guest MMU 检查映射；合法 RAM 映射可补缓存，MMIO 仍走设备回调，翻译或权限失败则按架构类型生成 guest fault；翻译得到的 guest 物理地址再按 `MemoryRegion` 分流为 RAM 直通或 MMIO 回调。整条路径说明：软件虚拟化的访存开销集中在 TLB 命中检查与偶发的翻译慢路径上，而语义正确性依赖对页表与权限的忠实解释。

1.3　中断虚拟化：从设备事件到 guest 异常入口

没有任何真实 RISC-V 中断控制器的条件下，异步中断靠一条贯穿软件各层的事件链实现，它从设备状态一路走到 guest 的中断处理程序。事件源可以是串口收到字符、块设备完成请求，也可以是虚拟定时器到期，它们都表现为设备模型内部的一次状态更新。控制器层由 PLIC 与 CLINT 两个模型承担：PLIC 维护外部中断源的优先级、使能与 pending 位，并提供 claim/complete 接口；CLINT 提供所选平台的本地定时器与软件中断能力 [3]。

通知路径是分层的。设备模型通过 `qemu_irq` 电平把事件递给控制器，PLIC 在 `sifive_plic_update` 里按优先级、使能、未 claim 三个条件逐 hart 挑选应上报的中断，再用 `qemu_set_irq` 拉高该 hart 的外部中断输入[8]。中断输入的变化最终反映到 CPU 的 pending 状态：`riscv_cpu_update_mip` 更新 `mip` 寄存器并调用 `cpu_interrupt` 置起 `CPU_INTERRUPT_HARD`[9]。此时若 vCPU 正忙于执行翻译代码，还需要 `tcg_handle_interrupt` 在非本线程上下文中调用 `qemu_cpu_kick` 唤醒或打断该线程[10]。

投递路径则是架构层面的决策。返回公共执行循环后，`cpu_handle_interrupt()` 按需调用 RISC-V 目标的 `riscv_cpu_exec_interrupt`，它读取 pending、屏蔽位、委托关系

[7] QEMU: `accel/tcg/cputlb.c`，`tlb_index()`；`docs/devel/tcg.rst`，代码失效说明。

[8] QEMU: `hw/intc/sifive_plic.c`，`sifive_plic_update()`。

[9] QEMU: `target/riscv/tcg/cpu_helper.c`，`riscv_cpu_update_mip()`。

[10] QEMU: `accel/tcg/tcg-accel-ops.c`，`tcg_handle_interrupt()`。

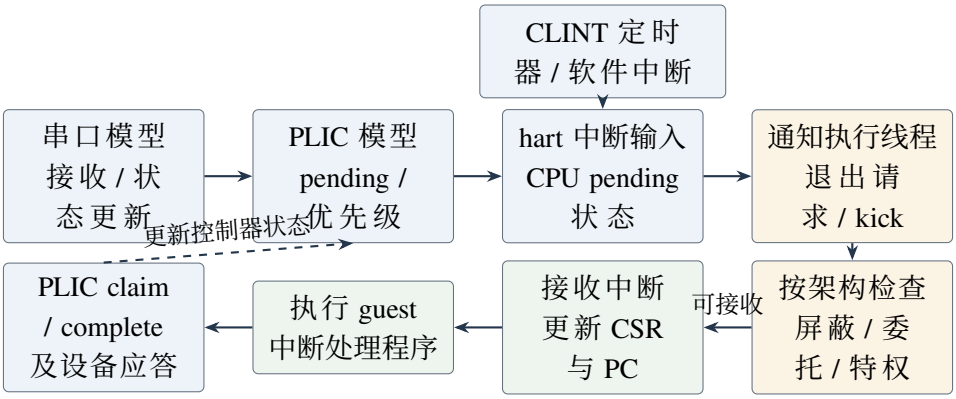


图 3　QEMU 中断生成与投递。外部中断以 UART/PLIC 为例；本地定时器和软件中断不经过 PLIC。置 pending、通知线程和 guest 接收中断是不同动作。[3]

与当前特权级，判断该 hart 此刻是否应接收某个中断；应接收时置起异常索引并调用 `riscv_cpu_do_interrupt`，由后者记录架构规定的异常返回信息、更新异常相关 CSR、切换特权级并改写 PC，进入 guest 的中断处理入口[11]。把事件置 pending、打断翻译代码执行、进入架构异常这三步分开是理解本节的关键：置 pending 是设备与控制器的状态变化，kick 是宿主线程调度层面的打断，架构投递才是 guest 状态机的一次转移。TB 链仍须响应异步退出请求。通常，`gen_tb_start()` 在生成代码入口插入一个轻量退出检查，通知路径改变相应状态，使后续代码返回公共循环，再做完整的架构中断判断；`CF_NOIRQ` 等特殊执行设置可以抑制该检查。因而“检查是否退出”与“完整仲裁并注入 guest 中断”是两层工作，不能用每 TB 全量轮询或完全不检查来概括[12][3]。

本地定时器使用 QEMU 的虚拟时钟作为软件时间源。所选 CLINT 兼容布局复用 ACLINT 的定时器实现：读取时间时将 `QEMU_CLOCK_VIRTUAL` 换算为设备计数，写比较器时更新到期事件；比较条件已满足则立即置起定时器中断，否则在回调中置位[13]。这种计时模型不等于默认保证确定性重放。宿主唤醒机制与 guest 中断身份仍是两个不同层面的对象 [3]。

图 3 以串口接收为例给出时序：字符到达 →PLIC 置 pending 并选优 →hart 中断输入拉高 →TCG 退出或 kick→CPU 判断接收 →guest handler 执行 →claim/complete 回写控制器。这条链把三个层次的介入点都标了出来，说明软件中断虚拟化的代价主要分布在控制器状态维护与每次投递判断上。

1.4　设备虚拟化：MMIO、软件 DMA 与后端服务

一个设备除了寄存器本身，还必须模拟三个方向的行为：CPU 到设备的控制路径、设备到内存的数据路径、设备回 CPU 的完成通知。

控制路径由 MMIO 分发实现。guest 对设备寄存器所在地址的读写，先经物理地址空间定位到 MMIO 区域，再转交 `MemoryRegionOps` 里注册的回调。以 NS16550 兼容串口为例，`serial_mm_write` 把写值裁到 8 位后送入 `serial_io_ops`，寄存器写会改变内部状态

[11] QEMU: `target/riscv/tcg/cpu_helper.c`，`riscv_cpu_exec_interrupt()`。

[12] QEMU: `accel/tcg/translator.c`，`gen_tb_start()`；`accel/tcg/tcg-accel-ops.c`，`tcg_handle_interrupt()`。

[13] QEMU: `hw/intc/riscv_aclint.c`，`cpu_riscv_read_rtc_raw()`、`riscv_aclint_mtimer_write_timecmp()` 与 `riscv_aclint_mtimer_cb()`。

并可能触发字符后端动作，寄存器读则返回当前状态[14]。也就是说，MMIO 分发把一次 guest 设备寄存器写转换为对设备状态机的调用，而不是先对某个真实 MMIO 地址进行宿主写操作，guest 看到的只是一个” 存在寄存器” 的串口，后端连的是字符设备、管道还是文件由 QEMU 配置决定。

数据路径以 virtio-blk 展开。guest 通过 virtqueue 提交描述符链，QEMU 在 `virtio_blk_handle_vq` 里取请求、解析描述符，把块后端处理结果写回 guest 内存，最后在 `virtio_blk_req_complete` 里写状态字节、`virtqueue_push` 归还描述符并 `virtio_notify` 通知 guest[15]。这里的数据搬运是软件 DMA：设备通过自己的 `AddressSpace` 访问 guest 内存，直接使用 guest 物理地址到宿主缓冲区的映射，并不自动套用某个 guest 进程的 CPU 页表；只有当配置了 vIOMMU 时，设备地址才会额外经过一次设备地址转换 [3]。区分” 软件 DMA 走设备地址空间” 与”CPU 访存走 CPU 页表” 是理解设备与内存两层如何分工的要点。完成通知则是回接上一节：请求完成后设备更新队列状态，需要时拉起中断，走 PLIC 与架构投递回到 guest。

最后要区分两种设备接口。模拟现有硬件接口（如 NS16550、PCI 设备）与 virtio 这类半虚拟化设备接口，都能由 QEMU 的软件后端提供服务，差别只在 guest 侧协议与性能；virtio 不是 CPU 硬件虚拟化指令，也不要求 CPU 必须用 KVM 执行，它只是约定了一套更利于软件实现高效 I/O 的接口。具体到 virtio-blk，virtqueue 组织请求与完成信息。以 split virtqueue 为例，guest 写好描述符链、更新 avail 环并按通知规则发出 kick，设备侧读描述符、执行后端、把结果写回 guest 内存、再写 used 环并按协商的通知规则决定是否产生中断，二者通过共享内存环而非真实总线握手来传递请求与完成状态。两者在” 由谁实现设备、guest 如何发现与驱动” 上不同，在” 都是软件模型” 这一点上相同。

1.5　能力与成本

把四个维度并置，才能看出软件方案” 能做什么” 与” 为什么慢” 是同一件事的两面。翻译代码让 guest 计算得以进行，软件地址转换让 guest 访存保持页表语义，事件状态机让异步中断成立，设备模型让 I/O 可用，四者组合起来才足以运行一套完整的 guest 操作系统；缺任何一环，其余环都没有意义。这是软件虚拟化的完整能力边界：它不依赖宿主硬件的任何虚拟化特性，因而可移植性极强——增加目标平台主要涉及相应 CPU/设备模型和宿主执行后端，而不必为每个 guest-host 组合重写整套实现 [2, 3]。

成本则须逐项拆解，而不是笼统写” 慢若干倍”。翻译与缓存维护的代价集中在冷路径（首次翻译）与失效路径（映射或代码变化），热路径只付出块链接的跳转与状态检查；访存代价来自每个访存前的软件 TLB 检查，以及 miss 时重走 guest 页表的开销；helper 调用打断了宿主代码的寄存器分配局部性，并在 C 代码与生成代码之间切换上下文；中断与设备 I/O 还引入 vCPU 线程与事件线程之间的同步，以及后端真实 I/O 的等待。这些成本落在不同频率与不同路径上：一个纯计算密集的 guest 主要承受翻译与寄存器分配的摊还成本，一个 I/O 密集的 guest 则主要承受 MMIO 回调、中断往返与后端等待，脱离具体工作负载无法给出统一的性能倍数——本综述亦不采信任何未指明配置与基线的性能数字。

[14] QEMU: `hw/char/serial-mm.c`，`serial_mm_write()`。

[15] QEMU: `hw/block/virtio-blk.c`，`virtio_blk_handle_vq()`、`virtio_blk_req_complete()`。

这套方案的边界也就此划清：它以软件实现架构语义，并在翻译执行、地址转换与中断投递中承担额外管理工作；功能等价依赖各模型及其组合的正确性，而不能仅由“纯软件”这一实现方式保证。下一章考察的硬件辅助，正是把这些高频路径上的软件介入下放到处理器与系统结构，但设备功能与管理策略仍需软件完成，硬件并不会自动补上软件方案中” 模型” 那一半。

2　硬件辅助虚拟化：Arm 架构与 KVM

本章把执行载体换成具备硬件虚拟化能力的 AArch64 平台。研究对象是 AArch64 guest 运行在 AArch64 host 之上，CPU 架构以 Armv8.6-A 手册为准；GICv3/v4 与 SMMUv3 是各自独立的系统组件规范，其实现与否由具体 SoC 决定，不能由 CPU 的 ISA 版本直接推定某平台必然具备全部功能 [5–7]。实现参照为 Linux 7.0.14 源码树中的 KVM/arm64、VGIC 与 SMMUv3 驱动 [4]。与前章纯软件方案的根本差别在于：guest 的绝大多数指令与访存由真实处理器直接执行，hypervisor 不再解释每条指令，而是退到异常级 EL2 上，只在受控操作或外部事件发生时介入。软件与硬件之间的分工因此重新划分——上一章由软件完成的敏感指令捕获、地址翻译、中断注入，本章部分由处理器与系统组件承担；但设备功能、资源管理策略与组合仍留在软件。下面仍沿 CPU、内存、中断、设备四个维度展开，最后用一次直通设备 I/O 把四者串起。

2.1　CPU 虚拟化：EL2、选择性陷入与 vCPU 上下文

Arm 虚拟化的基本布局由异常级（Exception level）刻画。guest 的内核运行于 EL1、其应用程序运行于 EL0，二者合起来构成一个虚拟机（virtual machine）；hypervisor 运行于更特权的 EL2，负责在多个虚拟机之间切换 [5]。EL2 为 guest 管理提供独立的控制寄存器与异常入口。guest 仍以 EL1 的接口建立页表、处理异常；它可以知道自己位于虚拟机中，也可以在大部分路径上无需感知 hypervisor，是否知情并非资源隔离的前提 [5]。这一布局回答了” 谁控制谁”：guest 控制的是 EL1/EL0 这一份状态，hypervisor 通过 EL2 控制系统资源的分发与回收，并为每个虚拟机分配一个 VMID 以在有关地址转换缓存中区分不同虚拟机的转换上下文。

让 guest 直接执行、又保留 hypervisor 控制权的关键是选择性陷入。EL2 提供一组控制寄存器决定哪些操作在 guest 态直接生效、哪些操作产生异常并送入 EL2，其中枢是 `HCR_EL2`。它逐位配置各条陷入路径，例如 `HCR_EL2.VM` 打开 Stage-2 内存翻译（详见下节），若干陷阱位决定 guest 访问某类系统寄存器、执行某条指令时是否陷入。于是敏感操作不必像软件方案那样由翻译器逐条识别，而是由硬件在指令执行处直接判定并转交 EL2。反过来，EL2 也能把少量标识寄存器对 guest 虚拟化：guest 读这些寄存器得到的是 hypervisor 预设的虚拟值，从而向 guest 呈现一个与其真实宿主不尽相同的处理器面貌 [5]。这些机制允许普通执行路径保留原生指令，而把需要管理的操作集中到受控的异常与状态虚拟化接口，避免软件翻译全部 guest 指令。

一次完整的运行闭环由硬件与软件分段完成。异常进入目标异常级时，硬件只自动保存恢复所必需的最小信息：处理器的当前状态（PSTATE 及各中断屏蔽位）存入 `SPSR_EL2`，异常首选返回地址存入 `ELR_EL2`，同步异常或 SError 的原因码写入 `ESR_EL2`[5]。也就是

说，硬件记录的是” 从哪来、为什么来、处于什么状态” 这组返回线索，并不代管通用寄存器。KVM 在 kvm_arch_vcpu_ioctl_run 里沿 KVM_RUN ioctl 组织起 vCPU 的完整生命周期：用户态（QEMU 进程）发起运行请求，内核经 kvm_call_hyp_ret 把 vCPU 上下文交给 EL2 的 __kvm_vcpu_run，后者逐批恢复 guest 的通用寄存器与系统寄存器，最后执行 eret 把控制权从 EL2 交回 EL1/EL0[16]。guest 触发被配置为退出的操作或事件后，处理器进入 EL2 异常向量，KVM 的入口代码保存必要状态并进入相应退出路径。KVM 先区分退出类型；同步异常等需要进一步解码的情况再使用 ESR_EL2，不能用旧的 ESR 值解释所有 IRQ 退出。硬件保存的那组最小信息与 KVM 软件维护的完整 vCPU 上下文之间因此存在明确分工：SPSR/ELR/ESR 是硬件保证的返回线索，GPR、浮点及系统寄存器状态的维护由软件负责，并可采用按需保存或延迟切换等优化，不能笼统说” 所有现场均由硬件自动保存”[4, 5]。

宿主自身的运行方式可再优化。VHE（Virtualization Host Extensions）允许 host Linux 内核本身运行在 EL2，并提供适合宿主内核使用的寄存器访问与 EL2&0 翻译体制；nVHE 则把 host Linux 留在 EL1，由独立的 EL2 hyp 代码执行敏感切换。二者改变的是宿主实现布局，guest 仍运行于 EL1/EL0。类似地，FGT（细粒度陷阱）与计数器虚拟化提供更细的选择性陷入与虚拟定时能力，用以进一步减少不必要的退出。这些属于对同一机制的工程化补充，本章不展开成特性年表，只指出它们的共同目标：减少不必要的状态切换与 guest 退出 [4, 5]。KVM/ARM 2014 年提出 split-mode 设计时的动机，正是要把 hypervisor 拆成” 跑在更特权的 Hyp 态的最小核心” 与” 复用 Linux 内核的大部分功能” 两块，从而不必从零维护一套驱动栈 [8]；这一历史背景解释的是 KVM 为何选择内核承载 hypervisor，而非本章所涉 Armv8.6-A 机制的实证来源，当下的具体机制仍以手册与源码为准。

2.2　内存虚拟化：VA 到 IPA 到 PA 的两阶段翻译

$$\mathrm{VA} \xrightarrow[\text{guest 管理}]{\text{Stage-1}} \mathrm{IPA} \xrightarrow[\text{hypervisor 管理}]{\text{Stage-2}} \mathrm{PA}. \tag{1}$$

guest 可以自由改写自己的页表，却无法越过 hypervisor 划定的内存边界，原因在于访存翻译被拆成两位管理者各自负责的一段。地址关系是三层：guest 虚拟地址（VA）经 Stage-1 翻译为中间物理地址（IPA），IPA 再经 Stage-2 翻译为真实物理地址（PA）。guest 操作系统管理 Stage-1，它把 IPA 当作自己的” 物理地址”，据此建立页表、分配页面；hypervisor 管理 Stage-2，把各 guest 的 IPA 映射到真实物理内存 [5]。Stage-1 的控制结构是 guest 侧的系统寄存器——TTBR0_EL1/TTBR1_EL1 给出两个 VA 区域的页表基址（Linux 通常将其分别用于用户与内核地址区域），TCR_EL1 给出页表粒度与地址空间布局；Stage-2 的控制结构则在 EL2——VTTBR_EL2 给出 Stage-2 页表基址，VTCR_EL2 给出其粒度与输出地址宽度。HCR_EL2.VM 是总开关，使 EL1&0 翻译体制下的 guest 访问受到 Stage-2 约束。这里的边界约束来自 Stage-2 的权限与属性位：即使 guest 在自己的 Stage-1 里把某页标成可写、可执行，hypervisor 仍可在 Stage-2 里限定为只读或不可执行，最终生效的是两阶段权限的交集。更关键的是，guest 页表自身的访存也走 Stage-2——硬件遍历 guest 的 Stage-1 页表

[16] Linux: arch/arm64/kvm/arm.c，kvm_arch_vcpu_ioctl_run()；arch/arm64/kvm/hyp/entry.S 中的 guest 寄存器保存/恢复与 eret。

时，页表项所在的地址也是 IPA，读取页表项本身须先经过 Stage-2，因此 guest 无法通过把某块 IPA 映射出去的方式窃取未分配内存。只有 EL1&0 翻译体制支持 Stage-2，EL2 自身访问使用独立的 EL2 翻译体制，不沿用 guest 的两阶段翻译上下文 [5]。

两阶段组合翻译并不意味着” 免费”。当所需转换未在缓存中命中时，处理器需要执行页表遍历（page walk）；Stage-1 页表项的读取还可能触发 Stage-2 遍历，因此代价并非简单地串行查两张表；把翻译结果缓存进 TLB 后，命中路径才接近单次直接访存的成本。TLB 条目按地址空间标识区分：Stage-1 的 ASID 区分同一 guest 内的不同地址空间，VMID 区分不同虚拟机，使多 guest 切换时不必整表失效。映射变化时仍需精确或整表式的 TLB 失效维护，这一维护动作由软件发起、由硬件执行。因此与 QEMU 每次访存执行软件 TLB 检查的路径相比，硬件辅助把转换缓存查询与未命中遍历交给 MMU，但两阶段遍历与失效维护的代价并未消失，只是从软件高频路径转移到了硬件分页单元与偶发的慢路径上 [4, 5]。

缺页与 MMIO 的处理由 KVM 的 `kvm_handle_guest_abort` 承接，它依据 `ESR_EL2` 里的故障分类把异常分流。对合法 guest RAM 的 Stage-2 缺页，KVM 沿 memory slot 把 IPA 换算成用户态提供的宿主虚拟地址后备，再填充 Stage-2 页表完成映射；这一路径上 guest Stage-1 缺页与 Stage-2 缺页的责任是分开的——正常的 Stage-1 fault 通常直接由 guest 的异常处理程序接收，无须先退出给 KVM；Stage-2 fault 才需要 hypervisor 处理。KVM 在其他异常条件下也可能向 guest 注入异常，二者不应混淆[17]。对 MMIO 区间则另走一路：guest 对设备寄存器 IPA 的访问没有对应内存后备，KVM 先尝试用内核的 `kvm_io_bus` 在宿主侧完成读写，成功则不再惊动用户态；只有内核侧无人认领该地址时，才把 `exit_reason` 置为 `KVM_EXIT_MMIO` 连同地址与数据一起返回用户态[18]。因此”guest 退出” 与” 回到 QEMU” 是两件事：一次 MMIO 访问若被内核的 io bus 处理，vCPU 全程不离开内核，也就谈不上每次都回 QEMU。这再次把” 何时介入、介入到哪一层” 写进了具体路径。

2.3　中断虚拟化：GICv3 的虚拟接口与 GICv4 的直接注入

中断侧的硬件辅助由一个独立于 CPU ISA 的组件承担。GICv3 按功能分成四块：Distributor 主要管理共享外设中断及全局分发控制，Redistributor 以每处理器为单位维护 SGI/PPI 及 LPI 的相关配置与 pending 状态，CPU interface 负责向本处理器呈现最高优先级中断并承载应答、去激活等接口，ITS 则把设备通过内存写发送的消息型中断（MSI）翻译成 LPI 并路由到目标处理器 [6]。这里的角色分工需要与内存、设备两维划清：ITS 处理的是中断路由，SMMU 处理的是 DMA 地址翻译，二者名字相近、对象不同，不能混为一谈。

GICv3 本身已经具备虚拟化支持，并不需要等到 GICv4。当 EL2 存在且启用时，hypervisor 通过一组 `ICH_*` 系统寄存器在 EL2 配置虚拟 CPU interface 与待注入的虚拟中断：`ICH_LR<n>_EL2` 这一组 List register 存放当前 vCPU 待处理、活动中的中断条目，`ICH_HCR_EL2` 控制虚拟接口的总开关与维护中断的产生条件，`ICH_VMCR_EL2` 提供虚拟接口的优先级屏蔽、EOI 模式等控制状态，`ICH_VTR_EL2` 则报告 List Registers 数量等实现能力。guest 则照常通过 `ICC_*_EL1` 与” 它的”CPU interface 通信——确认、处理、完成中断；

[17] Linux: `arch/arm64/kvm/mmu.c`，`user_mem_abort()` 与故障分流。

[18] Linux: `arch/arm64/kvm/mmio.c`，`io_mem_abort()` 及 `KVM_EXIT_MMIO` 的构造。

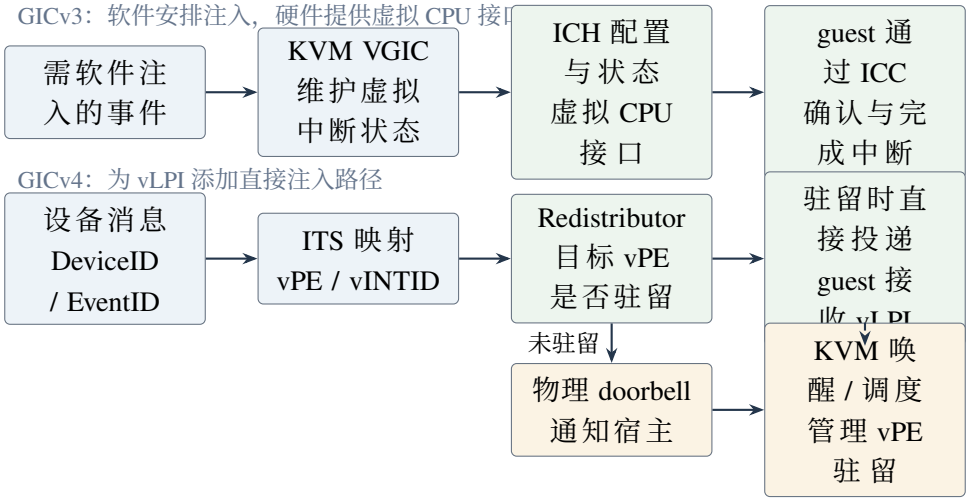


图 4　GICv3 与 GICv4 投递路径的机制对照。GICv4 并非虚拟化支持的起点，vLPI 直接注入也不消除配置和调度工作。[4, 6]

`HCR_EL2.IMO/FMO` 决定 guest 访问这些寄存器时落在虚拟接口还是物理接口 [6]。关键机制是：物理中断到达后，hypervisor 若判定其目标是当前 vCPU，就把对应虚拟中断写入 List register，硬件随后把其中最高优先级者按虚拟信号呈现给 EL1 的 guest，guest 的处理流程与裸机几乎一致；List Registers 数量有限，软件需要管理未装入的中断状态，并在合适时机装入或回收寄存器条目；其中还包括 active 状态，不能只把它理解成待处理中断队列 [6]。KVM 的 VGIC 正是这条软件一侧的实现：它维护每个 vCPU 的虚拟中断状态，在 vCPU 进入前把顶端条目写入 List register、退出时回读硬件状态折叠进软件模型，对 Distributor/Redistributor 等内存映射接口的 guest 访问则仍需以 Stage-2 缺页方式软件模拟 [4, 6]。所以 GICv3 的虚拟化是” 硬件做注入与优先级的临界路径，软件做状态管理与控制器 MMIO 模拟” 的组合。

GICv4 的增量限定在一个具体对象上：vLPI 的直接注入。GICv3 下 LPI 只能由 hypervisor 软件把 vINTID 写进 List register 来呈现；GICv4 让 ITS 直接向正在驻留的目标 vPE 呈现虚拟 LPI，不经 hypervisor：vLPI 的配置与 pending 信息存于按 vPE 组织的内存表（由 `GICR_VPENDBASER/GICR_VPROPBASER` 指向），ITS 依据翻译表里的 vPEID 找到目标，Redistributor 把 vLPI 与 List register 内容合并选优后直接注入 [6]。但这条直达路径有前提：目标 vPE 必须已调度到某物理 PE 上。当 vPE 未驻留时，控制器根据 ITS 映射中配置的物理 LPI 产生 doorbell，通知 hypervisor，由 hypervisor 决定调度或唤醒，KVM 侧在 `vgic_v4_doorbell_handler` 里标记 pending 并向该 vCPU 发请求[19]。换言之，GICv4 只把” 已驻留 vPE 的 vLPI 注入” 这一热路径下放到硬件，映射配置、vCPU 调度、未驻留时的唤醒与控制面仍由软件完成，不能表述为” 所有中断都完全绕过 hypervisor” 或”GICv4 才引入虚拟化”[6]。

图 4 把两条注入路径并列：GICv3 一侧，物理中断到达后由 hypervisor 决策并写入 List register，再经虚拟 CPU interface 呈现给 guest；GICv4 一侧，vLPI 由 ITS 直接注入驻留 vPE，未驻留时转入 doorbell 分支唤醒 hypervisor。两类中断最终在虚拟 CPU interface 进行优先级仲裁，但直接注入的 vLPI 来自 Redistributor 的内存表路径，无须先由软件写

[19] Linux: `arch/arm64/kvm/vgic/vgic-v4.c`，`vgic_v4_doorbell_handler()`。

入一个 List Register；管理这些表与 vPE 驻留仍需要 hypervisor。

2.4　设备虚拟化：SMMUv3、DMA 隔离与设备直通

CPU 已被 Stage-2 约束，仍需要 SMMU 的原因在于 DMA 这条绕过 CPU 的访存路径。CPU 访存经 VA→IPA→PA 两阶段翻译，而 DMA 由设备发起，其地址由设备侧驱动填充，走的是与 CPU 完全不同的通路，CPU 的页表与 Stage-2 对此鞭长莫及。若不加约束，一个被直通给 guest 的设备可能访问不属于该 guest 的物理内存，从而击穿内存隔离。SMMU 即设备侧的地址翻译与访问控制单元：它拦截设备发出的 DMA 事务，对其地址做与 CPU 类似的翻译与权限检查，再把翻译后的地址放回互联上 [7]。

翻译的定位由 StreamID 驱动。每个发起事务的客户端带一个旁路标识 StreamID，SMMU 用它索引 Stream Table，选中该设备对应的 Stream Table Entry（STE）；STE 决定这条流是否使能、是否经 Stage-1 翻译、是否经 Stage-2 翻译及相应页表基址 [7]。两级翻译与 CPU 侧语义对应：设备侧 Stage-1 把设备地址（即 guest 眼中的 IOVA 或设备地址）翻成 IPA，可由 guest 操作系统管理；Stage-2 把 IPA 翻成 PA，由 hypervisor 管理。当启用 Stage-1 时，STE 再指向一组 Context Descriptor（CD），请求携带的 SubstreamID（在 PCIe 中即 PASID）进一步选择不同 CD，从而为同一设备的不同进程上下文提供不同 Stage-1 翻译；SubstreamID 只影响 Stage-1 的选择，不影响仅由 StreamID 决定的 Stage-2[7]。配置表与命令队列由软件建立或提交，SMMU 读取并执行；事件队列和 PRI 队列则承载硬件向软件报告的信息。不同队列的生产者、消费者方向不同，不能一概说成软件写、硬件读。

这一结构给出若干层次的取舍，不能想当然地全盘启用。最简的直通只需 Stage-2：hypervisor 在 STE 里配置 S2 页表，把设备使用的 guest IPA 翻译成 host PA，guest 完全不必感知 SMMU 的存在，也就谈不上 vIOMMU。仅 Stage-2 是架构许可的一条路径，但它不等同于 Linux 普通直通域的实际默认——本机 SMMUv3 驱动为普通域分配 paging domain 时，在硬件支持 Stage-1 的情况下会优先选用宿主管理的 S1[20]，因而物理 SMMU 使用 S1，并不说明该 S1 是 guest 管理的，也不说明 guest 一定看见了 vIOMMU。guest 自管 I/O Stage-1 是可选项：只有当 guest 需要自己的 IOMMU 语义（例如在 guest 内部再隔离各驱动或做嵌套）时，才把 Stage-1 的 CD 与页表交给 guest 以 IPA 编址，由 hypervisor 的 Stage-2 继续把 IPA 翻成 PA，形成 S1+S2 的两级组合 [7]。

真正的设备直通是一项系统工程，SMMU 只是其中一环。把一块物理设备整个交给 guest，还须解决设备分配与控制接口、VFIO/IOMMU 的 DMA 映射、该设备中断在 GIC 中的路由（把物理中断映射到目标 vCPU）、以及设备复位与可迁移性等一组协作问题；SMMU 既不是设备模型，也不会替代 virtio 或实现网卡/磁盘的功能——它只解决” 设备访存不越界” 这一件事。反过来，软件设备的 guest 内存访问由宿主 CPU 通过后备内存映射完成，不要求经过物理 SMMU，也不一定每次都执行一次拷贝；后端若进一步使用真实设备，其硬件 DMA 仍由宿主的 IOMMU/DMA 管理机制约束 [4, 7]。

图 5 把本章三类访问与事件路径并置，它表示控制职责而非 SoC 的物理连线：CPU 访存走两阶段 VA→IPA→PA 翻译，设备 DMA 经 SMMU 的 STE/CD 选择后翻译，中断经

[20] Linux: `drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c`，`arm_smmu_domain_alloc_paging_flags()` 中的 S1/S2 选择。

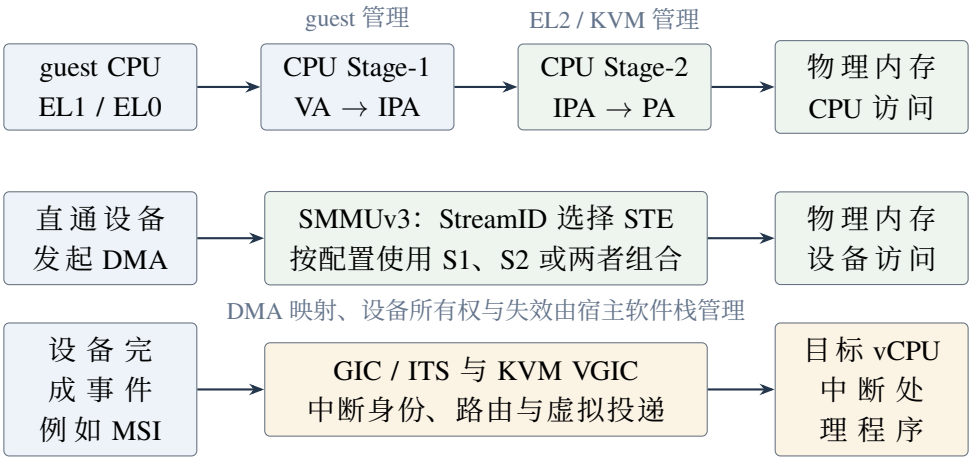


图 5　Arm 虚拟化的不同访问路径。CPU 地址翻译、设备 DMA 翻译和中断路由具有不同控制对象；图表示职责分工而非 SoC 互连布线，也不意味着所有 Linux 直通域都配置为 S2-only。[5–7]

GIC 的 Distributor/Redistributor/CPU interface 路由到 vCPU；图中所标出的 hypervisor 配置对象（Stage-2 页表、STE、`ICH_*` 与 List register）正是硬件机制的软件锚点。

2.5　协作小结：硬件负责机制，软件负责管理与组合

以采用 I/O Stage-2 的直通设备为例，guest 驱动通过已映射的 BAR 访问设备寄存器时，可以直接访问获授权的硬件区域，无须先退出到内核 MMIO 总线。配置空间或需拦截的访问才由软件处理。真实设备随后发起 DMA，SMMU 将设备使用的 guest 地址转换到获授权的 host 物理页；设备产生完成消息后，GIC 与 KVM 按配置投递中断。只有满足相应 GICv4 条件的 vLPI 才使用直接注入，其他事件仍有软件参与。普通 Linux 直通域也可使用宿主管理的 I/O S1，不能把这个示例当作所有部署的固定配置 [4, 6, 7]。

同一条 I/O 链中，CPU 执行隔离、内存边界、设备 DMA 权限和中断目标必须保持一致。处理器与系统组件加速频繁的执行、翻译和投递操作，而内存分配、映射建立、vCPU 调度和设备生命周期由软件组织。软件模拟、virtio 和设备直通因此可以在一台硬件辅助虚拟机内并存；它们改变的是设备服务路径，并不改变 hypervisor 对资源的最终管理职责。

3　嵌套虚拟化：从 Turtles 到 HyperTurtle

前两章分别考察了纯软件模拟与以 Arm 为代表的硬件辅助虚拟化，二者处理的都是”一层 hypervisor 直接管理一层 guest” 的问题。本章把对象再推进一层：当 hypervisor 自己就跑在一台虚拟机里时，机器状态、地址映射与事件策略要如何组合才既正确又可接受地快。本章的架构口径限定为 x86-64，即 amd64 这一 ISA 称谓，讨论的机制都落在 Intel 的 VMX 指令集、VMCS 控制结构与 EPT 扩展页表上，不使用 AMD SVM 的 VMCB/NPT 术语，也不把 Arm 的 EL2 异常级名称移植过来。两篇核心文献分别位于这条问题链的两端：OSDI 2010 年的 Turtles 回答了” 怎样在缺乏架构支持时正确实现嵌套” [9]，USENIX ATC 2025 年的 HyperTurtle 则回答了” 在已经能嵌套的前提下怎样减少不必要的跨层往返” [10]。前一问是后一问的前提，二者不能按同一标准横向比较性能，只能按” 谁控制

资源、关键路径怎么走、何时介入、代价多大” 这条主线先后展开。

3.1　问题建立：当 hypervisor 自己也在虚拟机里

嵌套虚拟化的层级用下标约定：L0 是跑在裸机上的 hypervisor，L1 是运行在 L0 虚拟机里的 guest hypervisor，L2 则是 L1 自己管理的嵌套 guest。L1 得到的机器接口必须包含它管理 L2 所需的虚拟化能力，即使真实 VMX 资源仍由 L0 控制；L2 则使用 L1 向它提供的虚拟机器接口 [9, 10]。

这一层次关系来自具体的部署需求。最直接的一类场景是租户自治：云厂商在 L0 上管理物理机，租户却希望在租给自己的虚拟机里再起 hypervisor，以便自行决定里面跑什么、如何迁移、何时快照，而无需惊动基础设施方。第二类是虚拟化软件自身的测试与开发——要复现”hypervisor 里再跑 hypervisor” 的复杂环境，单层虚拟化天然不够。第三类，也是 HyperTurtle 论文最关心的，是基于虚拟机的安全容器：Kata 这类运行时用 microVM 而不是共享内核的进程来承载容器，换取更强的隔离边界；当这一层容器运行时本身又跑在租户的 VM（L1）里时，microVM 就成了 L2，嵌套就不再是可选项而是部署形态本身。必须澄清的是，并非所有容器都需要嵌套——普通容器共享宿主内核，不构成 L2；只有” 在虚拟机内部再起 microVM” 这一形态才触发本节讨论的问题，因此不能把容器隔离一律等同于嵌套虚拟化 [10]。

问题的难点不在” 再多一层特权环” 这样的直觉，而在资源归属的不对称。L1 确实要使用虚拟化扩展：它要执行 VMX 指令、创建自己的 VMCS、配置自己的 EPT。但真正的硬件资源——处理器当前运行所需的 VMX 状态、活动 EPT 上下文与真实 IOMMU——都由 L0 独占控制。于是 L0 面临的任务不是给 L1 一个更强的 ring，而是向 L1 提供一个” 虚拟的虚拟化接口”：把 L1 要求的虚拟化操作接住、与 L0 自身对硬件和其余租户的约束合并，再组织成处理器当前可执行的一组硬件状态 [9]。

代价的直观体现是事件路径被拉长。当 L2 里发生一个需要 L1 处理的 VM-exit 时，CPU 先退出到 L0，L0 判断该事件需要向 L1 报告，并向 L1 反射退出，L1 处理完毕请求恢复 L2，L0 再次介入执行 VMEntry，最后才回到 L2，形成 $L2 \to L0 \to L1 \to L0 \to L2$ 的往返。但这条链不是所有事件都要走完：外部中断、NMI，以及只有 L0 在 VMCS02 里设置了 trap 而 L1 没在 VMCS12 里设置的事件，属于 L0 自己的范畴，L0 就地处理后直接恢复 L2，从 L1 的视角看这些事件根本不存在——正如单层虚拟化里 CPU 透明处理一次硬件事件后 hypervisor 继续运行一样 [9]。事件归属的判定权始终在 L0 手里，这才是”L0 保留最终控制” 的准确含义，而不是” 所有事件逐层转发”。

3.2　Turtles：把多层语义组合到单层硬件支持上

Turtles 是 KVM 之上的嵌套虚拟化实现，目标是运行未经修改的 guest hypervisor（KVM、VMware Server 等）及其 guest 系统。它在当时缺少专门嵌套硬件支持的条件下实现了可用的嵌套系统，并通过多类负载测量开销；论文的低开销结果不是对所有负载的统一上界。其实现涉及：CPU 上的 VMCS 合成、内存上的多维分页、I/O 上的多级设备分配，以及贯穿其中的退出归属判定 [9]。下面按四个维度分别说明” 谁读写什么、怎样合成、代价在何处”。

3.2.1 CPU：虚拟化 VMX 与合成 VMCS

在论文使用的单层 VMX 基线上，L1 在 VMX non-root 环境执行虚拟化管理指令会触发相应的拦截，这就给了 L0 一个天然的介入点：L0 捕获并模拟 L1 的 VMX 指令，也就是"trap-and-emulate"。大部分 VMX 指令的处理是"L1 退出到 L0、L0 按 VMCS 更新规则改写状态、再回到 L1"，让 L1 觉得指令真的执行了；但用来启动虚拟机的 `vmresume` 与 `vmlaunch` 是例外，因为此时 L0 必须模拟一次 L1→L2 的 VMEntry，流程变成"L1 退出到 L0，L0 直接 VMEntry 进入 L2" [9]。

状态合成的关键是三份 VMCS 的分工，如图 6 所示。VMCS01 是 L0 运行 L1 用的结构，从硬件角度看它和单层虚拟化完全一样：一个 hypervisor（L0）在 root mode 跑，一个 guest（L1）在 guest mode 跑。L1 为 L2 建立 VMCS12 来描述所需运行环境。该结构的地址与 host 状态具有 L1 视角下的含义，L0 不能不经检查、转换和合成就把它作为真实控制结构装载。于是 L0 以 VMCS12 为蓝本，再叠加 VMCS01 里对 L1 的约束，合成 VMCS02 作为真正运行 L2 的结构：guest state 大多可直接拷贝，host state 必须填成 L0 自己的、以便 CPU 从 L2 正确切回 L0，而 VMCS12 的 host state 要拷进 VMCS01 的 guest state，使 L0 模拟 L2→L1 切换时能加载正确的 L1 上下文 [9]。

控制数据的合成是正确性的重心。设想 L1 在 VMCS12 里要求 trap 某事件 E_A，但 L0 在 VMCS01 里并没有为 L1 trap 这个事件，那么为了把 E_A 转发给 L1，L0 必须在 VMCS02 里补上对应的 trap；同理，L1 用来向 L2 注入事件的字段、以及 CPU 用来报告退出原因的字段都要合并。这套合并的具体实现落在 KVM 源码里，论文没有逐字段展开 [9]。这里有一个必须澄清的术语边界：Turtles 所说的 VMCS shadowing 是 L0 在软件层面" 影子维护"VMCS12 并合成 VMCS02 的技术，与后来 Intel 在 Haswell 引入的同名硬件特性（让 L0 不必在每次 `vmread/vmwrite` 时都 trap）不是一回事；Turtles 的机制不依赖任何后续硬件能力 [9, 10]。同样，Turtles 描述的 2010 年合成逻辑，不应被当成当前 KVM 全部路径的写照——如今内核里的实现（如 `arch/x86/kvm/vmx/nested.c` 的 `prepare_vmcs02`）已经历多次重构并叠加了硬件加速，只可作为" 合成 VMCS02 这一思路延续至今" 的旁证，而非当年原型的源码 [4]。

3.2.2 内存：多维地址映射与合成 EPT

地址翻译在嵌套下多出一层：逻辑链是 L2 VA→L2 GPA→L1 GPA→host PA，即三层表。但当时的硬件 MMU 最多只支持两维（guest 页表 + EPT/NPT），无法同时走三层。Turtles 的做法是把三层折叠到硬件提供的两层上：L2 的常规页表保留下来负责 L2 VA→L2 GPA 这一层，L0 则合成一张 EPT02 直接完成 L2 GPA→host PA，而不是要求硬件逐个遍历 L2 页表、EPT12、EPT01 [9]。图 6 的上下两行分别给出逻辑映射与实际硬件执行路径，其中 EPT 下标" 管理层、被管理层" 表示该表由谁为谁维护，不表示地址转换方向。

这张合成 EPT02 是按需建立的。L0 以一张空 EPT02 启动 L2，当 L2 访问的 GPA 在 EPT02 里缺项时触发 EPT violation 退出到 L0；L0 检查 L1 维护的 EPT12 是否已有该地址的合法映射——有，则把 EPT12 里的 L2→L1 映射与 EPT01 里的 L1→host 映射拼接，写入 EPT02 的对应项；没有，说明这个缺页属于 L1 应负责的范围（例如 L1 尚未为该 GPA 分配内存），L0 就把退出交给 L1 处理，待 L1 更新 EPT12 后再回来补 EPT02 [9]。为保持 EPT02 与 EPT12 一致，L0 把 EPT12 所在内存设为只读，L1 一旦改写就 trap，L0 据此

同步 EPT02；同时 L0 还要 trap L1 的 `INVEPT` 以相应失效 EPT 缓存。这套机制相比两个朴素方案的优势在图 6 上体现为：shadow-on-shadow 要软件维护两层影子页表并压缩成 SPT02，代价最高；shadow-on-EPT 让 L1 用影子页表跑 L2，虽然用上了 EPT 硬件，但 L2 的每次页故障都要 L1 处理、每次页表写都要 L0 转发，等于把单层影子页表慢的原因在嵌套里又放大一遍。多维分页减少了影子页表维护引起的额外退出。EPT02 缺项或权限问题等仍可导致 EPT violation；L2 自身的普通页故障通常由 L2 处理，除非上层配置了相应拦截 [9]。此外 L0 还允许 L1 使用 VPID 给 TLB 条目打标签以避免每次切换都冲刷 TLB，并负责把 L1 自选的 VPID 重映射到 L0 的有效编号，防止两层各自随意取值时冲突；用大页给 guest 内存做后备还能进一步缩小 EPT 表 [9]。

3.2.3　中断：事件归属、转发与注入

嵌套下的中断必须分清三件不同的事：宿主自己产生的外部中断或 NMI，L0 要注入给 L1 的虚拟中断，以及 L1 希望注入给 L2 的事件。它们的处理路径不同。当 L2 正在运行且发生一次退出时，L0 首先按 VMCS12 与 VMCS02 的 trap 配置判断归属：属于 L0 的事件（外部中断、NMI，或只在 VMCS02 里 trap 的事件）就地处理后恢复 L2；属于 L1 的事件，L0 把 CPU 已更新的 VMCS02 字段拷回 VMCS12，再恢复 L1，让 L1 觉得是一次 L2→L1 的直接退出 [9]。注入方向上，L1 在 VMCS12 里写的注入字段要被 L0 合并进 VMCS02，才能让 CPU 真正把中断送进 L2。

设备分配场景揭示了一个关键的非对称性：DMA 直达不等于中断直达。多级设备分配让 L2 的数据可以直接落到设备，但中断在当时的 x86 架构上无法直接派发给 guest——中断到来总是先退出到 L0，L0 再转发 L1，L1 注入 L2，其中断的 EOI 写回也会再次触发退出。论文据此推断，设备中断及其 EOI 造成的退出是设备分配场景下残余开销的最大来源，并用一个去掉中断、纯轮询的网络驱动做了对照实验来支撑这一假设 [9]。需要强调的是，这是 2010 年原型在当时架构下的路径描述；后续硬件与 KVM 实现继续演进，因而不能把当年的中断处理路径断言为所有当前实现的必然行为 [9, 10]。

3.2.4　设备：多级设备分配与虚拟 IOMMU

两层虚拟化各可选用模拟、半虚拟化或设备分配三种 I/O 方式，组合出九种可能，Turtles 重点评估了其中五种，并把”L0→L1 与 L1→L2 都做设备分配” 称为多级设备分配 [9]。其目标不是把真实网卡” 直通” 给 L2 就完事，而是要在数据路径外补全三条辅助路径。

DMA 的地址问题是第一道坎：guest 驱动用的是 guest 物理地址，设备访存用的是 host 物理地址，而当时的 IOMMU 只支持一级翻译，两层映射必须再次压缩到一层。对可修改的 L1 可用半虚拟化 IOMMU——把 L1 里设置 L2→L1 映射的代码替换成对 L0 的 hypercall，L0 把映射里的 L1 地址换成对应 L0 地址后写入真实 IOMMU；对不改 L1 的方案，L0 为 L1 模拟一个 IOMMU，L1 以为自己在真 IOMMU 上建 L2→L1 映射，L0 截获这些映射、重映射 L1 地址、把 L2→host 的映射写到真实 IOMMU 上 [9]。中断路径如前所述仍需退出转发。MMIO 与 PIO 则相对干净：嵌套 guest 的 MMIO/PIO 与单层 guest 走同样的机制，不在 I/O 关键路径上引入额外退出 [9]。也就是说，多级设备分配的价值在于把 DMA 与 MMIO/PIO 留在直通路径上，代价集中在中断转发与 L1 注入这两类退出上，隔离和通知并没有被” 直通” 二字抹掉。

图 6 把 CPU 与内存两个维度的合成放在同一张图里：上排是 VMCS01/12/02 的控制

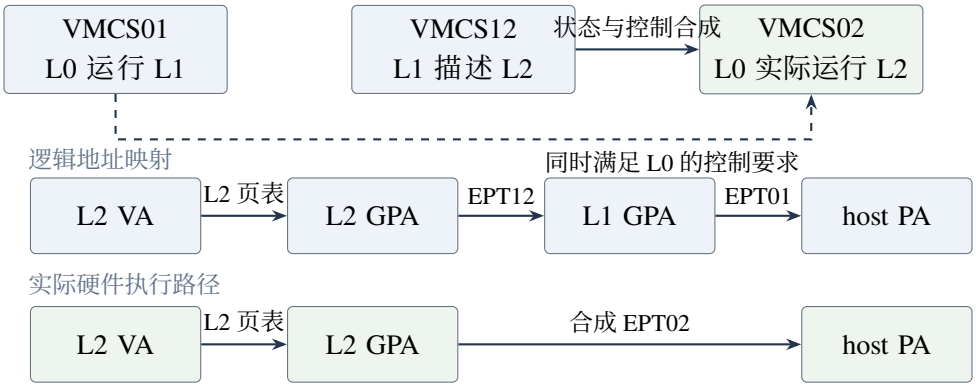


图 6　Turtles 的控制与地址映射合成。VMCS/EPT 下标表示管理层和被管理层；EPT02 由 L0 按需建立并维护一致性。[9]

合成关系，下排是地址映射的逻辑链与硬件执行链的对照。该图同时说明，Turtles 的” 组合” 本质上是把 L1 的语义要求（VMCS12、EPT12）作为输入，与 L0 自身的约束（VMCS01、EPT01）合并成一份硬件可执行的产物（VMCS02、EPT02），而合并的依据是退出归属与地址归属的判定规则。

3.3　从” 能嵌套” 到” 少往返”：性能问题的重新定位

Turtles 的成就是正确性与可行性：在不修改 guest hypervisor 的前提下给出了嵌套语义。但它的开销分析也暴露了问题的下一层形态。论文用 L2 里循环执行 `cpuid` 的合成微基准测单次退出的周期成本：裸机上 `cpuid` 约 100 周期，单层虚拟化约 2600 周期（其中约 1000 周期是模式切换），嵌套则飙到约 58000 周期 [9]。原因是 L1 每处理一次 L2 退出，都要执行多条 `vmread/vmwrite` 这类特权指令，而每条又各自触发一次对 L0 的退出——退出被逐级放大。论文在这里区分两类优化。L0 的合并路径可以只拷贝变化字段，并探索直接批量访问真实 VMCS 的方式；后一尝试不严格符合 VMX 规范，不能保证跨处理器适用。另一个称为 DRW 的评测配置则明确修改 L1，使其直接读写 VMCS12，用来估计省去 L1 的 VMREAD/VMWRITE 拦截的潜在收益。这两类直接访问不能混称为只修改 L0 的同一个优化 [9]。

HyperTurtle 的动机分析把这个现象测得更系统，给出两条明确的量化结论。其一，从 VM-exit 到下一次 VM-entry 的处理时长因退出原因而异，其中包含切换和软件处理，不能将整个数值视为纯硬件切换延迟：L0 处理一次 nop hypercall 从 L1 退出约 0.85μs，从 L2 退出则约 2.82μs；NMI 的差距更大，从 L1 是 5.7μs，从 L2 是 16.7μs [10]。其二，L2 触发的退出普遍比 L1 更贵，因为 L0 需要把退出注入给 L1、并维护合成控制结构及其他管理状态；论文援引 Vilanova 等人的结果指出，world-switch 相关开销约占一次 10μs 的 `cpuid` 模拟的 33% [10]。这两个结论共同指向：跨层往返及其处理是这些负载的重要成本来源，不能只按地址翻译层数解释嵌套开销。

于是出现一个张力。想绕过 L1 的优化确实存在——DVH（Direct Virtual Hardware）把 L0 的虚拟设备直接分配给 L2，从 L2 的数据路径上拿掉 L1，延迟与吞吐都明显改善。但代价是 L1 对 L2 失去了控制：L1 不再处于 L2 的网络控制路径上，无法再施加路由、监控、限速等策略，而这些正是容器管理所必需的 [10]。” 路径更短” 与” 管理权仍在租户” 由此对立起来。HyperTurtle 试图回答的过渡问题因此不是” 怎样把 L1 整个搬进 L0”，而

是：能否让一部分由 L1 定义的处理逻辑在 L0 就地执行，既省掉往返、又保留 L1 的策略控制，同时不破坏 L0 的资源边界与隔离。

3.4 HyperTurtle：把 L1 的部分处理逻辑下放到 L0

HyperTurtle 的答案是把 L1 hypervisor 的一部分功能以 eBPF 程序的形式下放到 L0，称之为 hyperupcall：L1 把自己对某类 L2 事件的处理逻辑写成一个 eBPF 程序，交给 L0 在 L2 事件的处理路径上就地调用，从而在"L1 仍然定义逻辑、L0 负责执行与兜底" 的前提下省去进入 L1 的往返。其设计目标有四条：消除多余 world-switch（性能）、保留 L1 对 L2 的策略控制（控制）、最小化对 L0 的改动（低侵入）、提供可推广到不同子系统的通用机制（通用） [10]。实现面向 KVM，且限定为单层嵌套。

3.4.1 通用机制：注册、共享状态、执行与回退

注册路径与执行热路径是两回事，必须分开看。安装由 L1 通过 hypercall 发起，经 L0 的 KVM 与 QEMU 等用户态组件落地：L1 提供编译好的 eBPF 程序与要挂接的 hook，L0 借 Linux 内核已有的 eBPF 注册机制完成验证、加载与链接；与更早的 hyperupcall 原型不同，HyperTurtle 把" 加载" 与" 链接" 拆成两个 hypercall，使同一个 hyperupcall 能挂到多个 hook 而无需重复加载 [10]。安装完成后，事件热路径可调用已链接的 eBPF：例如 L2 退出到 L0 后，就地完成可下放的处理，再恢复 L2；网络 hook 则位于相应的数据处理路径。图 7 对比的是普通处理与下放后的事件路径，安装动作不在图示热路径之内。

下放逻辑要在 L0 里正确运行，需要两类共享状态。一是数据通道：L0 与 L1 之间通过共享的 eBPF map 交换数据，而不是早先 hyperupcall 的基于数据包的方式，以支持并发访问；具体做法是新增一个 hypercall 创建一个共享内存 PCI 设备，把数组型 eBPF map 暴露给 L1。二是受限的内存访问：新增 helper `bpf_probe_read_hyperupcall` 让 eBPF 能读 L1 的物理内存，该 helper 会校验地址有效且确属发起该程序的 L1，从而把" 每个 hyperupcall 只能读自己所属 L1 的内存" 这一隔离写成强制约束 [10]。

程序验证只是信任边界的一部分，访问授权与状态同步同样需要单独设计。HyperTurtle 支持两种安全模型：一是 L1 提交的程序由 L0 的 eBPF verifier（或更严格的 Prevail 之类）验证；二是只接受云厂商预先审核、签名验证过的程序。无论哪种，隔离都由 verifier、受限 helper 的地址绑定、以及共享状态的同步协议三者联合保证，任何单独一环都不足以宣称跨租户安全 [10]。

最后，下放永远带着回退。HyperTurtle 明确保留了 L1 的原始处理路径作为兜底：内存池耗尽、fault log 写满、加锁失败、遇到不支持的情况时，一律回退到传统的、需要进入 L1 的处理方式 [10]。这意味着 hyperupcall 优化的是" 适用热路径" 而不是取消 L1；L1 仍是功能完整性与复杂功能的最终承担者。

3.4.2 主案例：EPT fault 处理

EPT fault 展示了同一逻辑操作引发多次跨层往返的情形。论文的 EPT hyperupcall 面向 shmem 文件后备内存，不能直接当作所有内存后备方式的通用处理器。图 7 已给出通用对比，这里补上具体的步数差异：L2 因 EPT02 缺项触发退出到 L0（第 1 步），L0 检查 EPT12 是否有合法映射（第 2 步）；若没有，L0 把 EPT fault 注入 L1 并进入 L1（第 3

步），L1 在 EPT12 里建立映射（第 4 步），L1 试图恢复 L2、又 trap 回 L0（第 5 步），L0 进入 L2（第 6 步），L2 重执行触发 fault 的指令——由于 EPT02 仍缺项，再次退出（第 7 步），L0 再次查 EPT12（第 8 步）、找到条目、用其补全 EPT02，最后再进入 L2（第 9 步）。一次缺页共产生 6 次 world-switch，而非嵌套下的 2 次 [10]。EPT fault hyperupcall 做的事，就是把这个" 缺项 → 进 L1 建映射 → 重执行 → 再缺项 → 再查表" 的循环里属于 L1 的那段，替换成 L0 内的一段 eBPF：在发现 EPT12 缺项时，hyperupcall 先检查已有映射和并发状态；适用时复用映射或从预分配内存池取帧，再更新 EPT12 与 fault log，L0 随即完成 fault 处理并恢复 L2，省掉了进入 L1 及其后的重执行往返 [10]。

正确性的难点全部在共享状态上。第一是内存来源：eBPF 上下文里分配内存过于复杂，因此 L1 维护一个预分配的帧池放在共享 map 里，hyperupcall 从池里取帧，池空则回退；实验表明 4096 帧足以避免同步补充，论文全文用这一池大小 [10]。第二是与 L1 的一致性：hyperupcall 改动了 EPT12，L1 必须知道；它通过共享的环形 fault log 记录自己映射过的 L2-PA，L1 在下一次进入 L1 的 vm-exit 时把这些条目对应的 L1-VA 映射进 VMM 地址空间；log 写满则回退。反过来，VMM（或 virtiofsd 等外部进程）可能先行建了映射而 hyperupcall 尚未察觉，因此 L1 把 VMM 的 cr3 拷进共享 map，让 hyperupcall 能用 `bpf_probe_read_hyperupcall` 遍历 VMM 页表、判断该地址是否已有映射；同时把 shmem 的页故障处理函数包一层，先加锁、再查 EPT12 是否已被 hyperupcall 映射、再走原处理、最后把对应的 L2-PA 写进环形 page fault log，以弥合"hyperupcall 已建映射但 L1 尚未看到" 的窗口 [10]。第三是并发：整个 hyperupcall 包在一把锁里，锁也经共享 map 暴露给 L1，使 L1 处理 EPT fault 时能取同一把锁；受 eBPF 锁实现限制，取锁失败即回退 [10]。核心评议因此不在"eBPF 跑得快"，而在" 减少跨层切换之后，如何仍正确维护两层内存管理状态"——池、双日志、cr3 共享与锁，本质上是把原本主要由 L1 内存管理路径维护的状态关系，改造成 L0 与 L1 之间可并发、可回退的协议。

3.4.3　扩展案例：网络策略与性能采样

网络案例回应的是 DVH 的取舍。DVH 把 L0 提供的虚拟网卡直接分配给 L2，数据路径短、性能好，但 L1 失去策略控制；HyperTurtle 的做法不是退回两层 virtio，而是让 L1 把过滤、限速、监控逻辑写成 eBPF 挂到 L0 的网络接口 hook 上，从而在保留 DVH 直通数据路径的同时恢复 L1 的控制。论文实现了四个示例程序：什么都不做的 pass、按源 IP 过滤的无状态防火墙、可运行时调参的限速器、以及监控 L2 全部 TCP 连接的 TCP-Top [10]。必须澄清的是，这里的" 分配" 是把 L0 的虚拟网卡（vNIC）交给 L2，而不是物理网卡直通；隔离靠" 每个 hyperupcall 只能跑在它所属 L1 拥有的虚拟接口上" 来保证，且每个 vNIC 可挂不同程序，使不同 L2 各受其 L1 策略约束。为接回 Kubernetes，论文实现了一个 CNI 插件做动态接口管理：pod 创建时经 hypercall 请求 L0 建 vNIC、由 QEMU 热插到 L1、CNI 再配置网络命名空间与地址，从而在 k8s 管理面下既用 DVH 又保留 L1 的网络函数 [10]。这个案例的意义在于展示通用性：hyperupcall 不局限于内存，还能把" 数据处理" 与" 策略控制" 重新拼在同一层。

profiling 案例则展示另一种下放形态：采样本身不需要 L1 提供信息，只需把样本送回 L1。普通情况下，从 L1 对 L2 采样一次，L0 注入 NMI、L1 处理采样并访问 MSR、`vmread/vmwrite` 改 VMCS、再执行 `vm-enter`，全程约 26 次 world-switch；其中 L0 注入

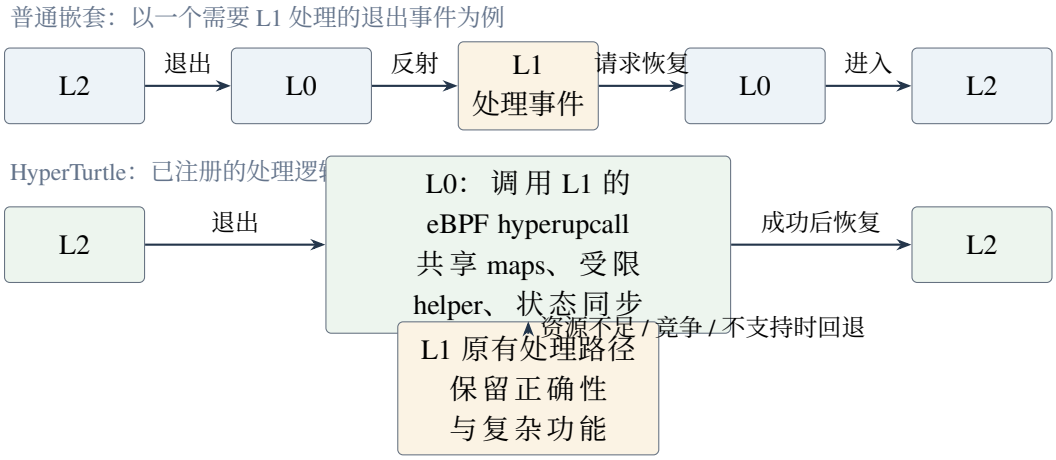


图 7　HyperTurtle 减少跨层往返的方式。图显示通用热路径，具体 EPT 缺页还可能包含重执行后再次退出等步骤；成功下放只减少特定处理所需的切换，不取消 L1。[10]

NMI 到 L1 要 7.8μs，模拟 L1 的 `vm-enter` 还要 8μs [10]。HyperTurtle 借 Linux 已有的 perf 接口把 eBPF 注册成采样事件，新增 helper `vcpu_probe` 让 hyperupcall 直接读 L2 的寄存器，样本经共享 map 的环形缓冲送回 L1；注册过程把采样限制到相应执行对象的 guest 状态，并结合受控寄存器读取接口维持访问边界 [10]。两个案例合起来说明方法通用，但结论有边界：它们证明的是” 特定处理路径可以减少往返”，而不是” 所有中断都能直接注入” 或” 所有设备都能完整下放”。

图 7 概括了 HyperTurtle 的通用形态：普通嵌套下，一个需要 L1 处理的事件要走 L2→L0→L1→L0→L2；HyperTurtle 下，已注册的处理逻辑在 L0 内以 eBPF 形式就地执行，成功后直接恢复 L2；资源不足、竞争或不支持时沿虚线回退到 L1 的原始路径。图中的成功恢复与回退两条分支，与上文 EPT 案例的步数拆解一致——注意 EPT 缺页还包含重执行后再次退出等步骤，成功下放只减少特定处理所需的切换，并不取消 L1 作为正确性与复杂功能兜底的存在。

3.5　实验证据评议

两篇论文的硬件、软件年代与实验目标不同，不能把其结果拼成统一的性能排名。Turtles 的宏观负载使用 KVM 同时充当 L0 与 L1，并启用多维分页。相对单层 guest，kernbench 的运行时间增加 14.5%，SPECjbb 分数下降约 7.8%；使用修改 L1 的 DRW 配置后，两项数值分别变为 10.3% 与 6.3%[9, §4.1, Table 2]。这些差异说明，管理路径密集程度会影响嵌套成本，摘要中的较低开销不能解释为适用于全部工作负载的上界。DRW 还改变了 guest hypervisor，必须与未修改 guest 的基线分别讨论。

HyperTurtle 进一步把热路径收益与端到端效果联系起来。表 2 只列出各项实验直接报告的比较结果，不将动机分析中的绝对延迟与另一个实验中的加速比拼接成推算值。EPT fault 的平均与尾延迟、服务启动时间、网络往返延迟和采样处理时间是不同度量，不能统一称为“系统加速”。

EPT fault 结果对应的是在 Kata 虚拟机内触发内存按需映射的微基准。减少进入 L1 的往返，使平均延迟接近论文中的非嵌套参考，但只有在负载频繁触发此类缺页时，收益才会显著传导。服务启动实验因此具有解释力：论文从发出启动命令开始计时，直到容器

表 2　HyperTurtle 的代表性评测结果（原论文报告，非本文复现）。EPT 采用正文 §5.1 的数值；图 9a 的平均改善标注为 5.26，二者存在末位差异，不影响此处机制比较。[10]

度量与条件	基线	报告结果	出处
EPT fault 处理	普通嵌套 L2	平均延迟改善约 5.25×，99p 改善 4.76×	§5.1，图 9a
Kata 服务启动	相同镜像的普通嵌套	4 KiB 页平均缩短 27%；2 MiB 页缩短 8%	§5.2，图 10
UDP 往返延迟	Nested-VirtIO	HT+Firewall 平均约 60.53 μs，基线约 90.2 μs	§5.1，表 7 及正文
采样处理时间，1000 Hz	L1 对 L2 的常规采样	HT 对 L2 的处理时间改善 7.15×	§5.1，图 9b

中的 Web 服务能够响应请求，而不是只计 guest 内核 boot。图 10 还区分了 4 KiB 与 2 MiB 页；使用大页时，Kata 在 L1 中仍以 4 KiB 页映射 L2 内核的部分路径，因而 hyperupcall 仍有作用，但端到端改善较小 [10, §5.1–5.2]。这组对照也提醒读者：页大小、后备内存组织和启动阶段都会改变缺页优化的收益。

网络结果须同时比较性能与控制能力。无 L1 策略的 Direct-Assignment 是性能参考，不能冒充功能完全相同的基线。论文用 HT+Pass 与 HT+Firewall 区分基础设施成本和策略程序成本：前者平均延迟约 55 μs，接近直接分配的 54.5 μs；后者增加到约 60.53 μs，但保留了 L1 的防火墙控制，并低于两层 virtio 的约 90.2 μs[10, §5.1]。由此可以支持“缩短路径而保留策略”的主张，却不能推导出所有 eBPF 网络策略都具有相同开销。profiling 的端到端实验则在相同直接分配网络上改变采样频率：4000 Hz 与 1000 Hz 下，论文报告满足 500 μs 查询延迟约束时的 Memcached 吞吐分别提升 26% 与 12%[10, §5.2, 图 14]。采样更频繁时，减少采样路径切换的价值更明显；这不是无采样负载的通用吞吐提升。

实验外推还受平台与系统配置约束。HyperTurtle 使用双路 Xeon Silver 4216，固定 vCPU 与 vhost 线程在同一 NUMA 节点内的不同物理核，关闭 SMT，并设置 `idle=poll` 与 `nopti`；吞吐压力实验还增加 Kata 的 vCPU 数量，以使单个 vhost 线程达到瓶颈 [10, §5, Table 6]。这些配置改变空闲等待、隔离开销或资源争用条件，不能据此替生产环境预先确定收益。实现规模也应按组成理解：论文 Table 5 分列 L0 KVM、eBPF 框架、QEMU 安装器与动态接口、L1 内存管理和 KVM，以及各个 hyperupcall 的代码量。只引用“KVM 改动 151 行”会漏掉其他必要部分 [10, §4.6]。

3.6　局限与可检验的后续问题

EPT hyperupcall 的边界在原型中十分具体：它不支持分配大页，对尚未映射进 L1 VMM 页表的部分 page-cache 页面不能直接处理，且全局 EPT hook 暂时只允许一个 L1 为一个 L2 使用该处理器。复杂 L1 功能的并发同步也限制了完整下放，因此原有 L1 路径必须保留 [10, §4.8]。这些是特定 EPT 原型的限制，不能扩大成“所有网络 hook 都只能服务一个 L2”；网络路径利用每个虚拟接口的 hook，有不同的实例组织方式。

下一步首先应测量一致性成本，而不只继续压缩最理想路径的延迟。可以逐步增加

L1 与 L2 的并发度，分别控制帧池补充频率、fault log 占用率和锁竞争，报告回退率、99p 延迟及宿主 CPU 时间。这样才能判断省下的切换是否被共享状态维护重新消耗。把 EPT hook 扩展为 per-L2 handler 后，还应检验多个 L1 同时注册、撤销与更新程序时的隔离和资源消耗。对生产适用性的评估，则需比较不同 SMT、NUMA、空闲等待和页表隔离设置，而不是假定论文配置代表所有云主机。

安全讨论应区分程序执行约束与系统级信任关系。验证器不能代替 helper 的权限设计、VM 归属检查和共享状态协议；可加载程序的执行时间、内存消耗与撤销机制也需要纳入审计。HyperTurtle 假定 L0 能够读取并处理 L1 的相关状态，这与把 L0 排除出信任边界的机密虚拟机模型不同，不能直接移植其安全结论。VM 迁移、动态策略更新和更深层嵌套还要求迁移或重新建立程序、maps 及 hook 的关系；这些工作既涉及性能，也涉及生命周期的一致性。

云租户自主管理虚拟机、Kata/serverless 与租户级网络策略为这种方法提供了应用场景。对国内云平台而言，值得评估的是既保留租户控制又减少嵌套管理往返的接口，而不是仅复制一个实验加速比。Turtles 展示了如何把多层语义落实到现有硬件，HyperTurtle 则在这一基础上重新安排部分处理逻辑的位置；其推广价值取决于下放后的状态同步、安全和运维成本能否保持可控。

结语

回到本文开头的问题：怎样让客户机拥有一台机器的使用接口，却不拥有对真实资源的最终控制权。三个部分给出的答案并不构成一条简单的替代链，它们是同一问题在不同约束下的三种实现。沿四维框架收束，可以得出三点判断。

第一，四个维度必须协作，任何单一维度都无法独立构成一台可用的虚拟机。CPU、内存、中断、设备各自解决执行、地址、异步事件与数据通路，而这些状态必须相互一致：地址转换的结果决定了访存与 DMA 是否落到正确的内存，中断投递要依赖 CPU 的执行上下文与设备的后端状态，设备完成通知又回到中断。软件方案里四者靠同一个进程内的数据结构连接，硬件方案里它们靠同一套异常级与系统结构的分工连接，嵌套方案里它们靠同一组跨层映射与事件归属连接 [3, 5, 9]。割裂地看任何一个维度，都看不到整台机器为何能够运行。

第二，硬件辅助减少的是高频路径上的软件介入，管理策略与设备语义仍然需要软件。Stage-2 把地址翻译下放给硬件，但内存后备的分配与映射维护仍是 KVM 的工作；在相应配置下，GICv4 能向驻留 vPE 直接注入 vLPI；映射配置、调度和 doorbell 响应仍需软件参与；SMMU 约束真实 DMA，却不能替代设备模型去实现网卡或磁盘的功能 [5–7]。硬件提供的是机制，把机制组合成服务并决定何时使用它，仍是软件的责任。

第三，嵌套虚拟化的本质是把资源边界与上层策略组合起来。Turtles 展示了将多层语义组合到现有硬件支持上的实现，跨层往返是其中的重要成本 [9]；HyperTurtle 的适用范围是被安全下放且一致性成本可控的热路径——它依赖 verifier、受限 helper 与共享状态的同步协议维持相应隔离边界，并以回退路径兜底，因此优化的是适用的热路径，不取消 L1[10]。

最后需要说明证据的边界。HyperTurtle 的评测在固定绑核、关闭 SMT、`idle=poll` 与

`nopti` 的收紧配置下取得，网络吞吐实验还调整了 vCPU 数量，这些配置限定了实验中的资源争用、空闲等待和隔离开销条件 [10]。因此论文报告的相对收益说明的是该配置下热路径优化的效果，不能直接外推为共享多租户生产环境的期望收益；本文未进行性能复现，对更广泛部署的适用性仍需通过独立实验检验。

把三部分并置，真正可比较的是执行、状态与策略被放在了哪里：软件把三者都放在进程内，硬件把机制下放而保留管理，嵌套则在资源边界与上层策略之间重新分配处理逻辑。虚拟化技术的演进，最终是在减少跨层操作与维持控制权之间不断寻找新的平衡点。

下表把三种方案在同一四维框架下并置，作为全文的收束。

维度	QEMU 软件	Arm 硬件辅助	x86-64 嵌套
CPU	翻译执行与软件 CPU 状态，guest 指令由宿主代码实现	guest 在 EL1/EL0 原生执行，EL2 控制，KVM 切换上下文	虚拟化 VMX 与合成 VMCS，按退出归属决定就地处理或转发
内存	软件 MMU 与 TLB，解释 guest 页表并映射到宿主内存	Stage-1 与 Stage-2 两阶段翻译，KVM 管理 Stage-2 映射与相关异常	合成 EPT 压缩多层映射，维护映射一致性并下放部分缺页处理
中断	软件控制器维护 pending，经架构异常投递到 guest	GICv3 虚拟接口安排注入，GICv4 对驻留 vPE 直接注入 vLPI	跨层事件转发与注入，减少特定处理路径的往返
设备	MMIO 设备模型与软件 DMA，后端完成请求并通知 guest	模拟、virtio 与直通三种形态并存，SMMU 约束真实 DMA	设备与 IOMMU 映射组合，下放数据处理与上层策略

表 3　四维框架下的三种实现对照

参考文献

[1] Gerald J. Popek and Robert P. Goldberg. Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, 17(7):412–421, 1974. doi: 10.1145/361011.361073.

[2] Fabrice Bellard. QEMU, a fast and portable dynamic translator. In *Proceedings of the 2005 USENIX Annual Technical Conference, FREENIX Track*, pages 41–46. USENIX Association, 2005. URL https://www.usenix.org/legacy/event/usenix05/tech/freenix/bellard.html.

[3] QEMU Project. QEMU source code and developer documentation, 2026. 源码版本：`v11.1.0-3334-g56afd82bd4`；commit `56afd82bd4db9688f1c1c9d0e2ea163b8cde3283`。重点参照 Translator Internals、The memory API 及 RISC-V virt 文档.

[4] Linux Kernel Community. Linux kernel source code, version 7.0.14, 2026. 重点参照 `arch/arm64/kvm/`、`arch/x86/kvm/` 和 `drivers/iommu/arm/arm-smmu-v3/`；具体函数见正文脚注.

[5] Arm Limited. *Arm Architecture Reference Manual, Armv8, for Armv8-A Architecture Profile*. Arm Limited, 2020. 本文 CPU 架构基准；参照 D1.5、D1 异常模型、D5 VMSA 及相关系统寄存器定义.

[6] Arm Limited. *Arm Generic Interrupt Controller Architecture Specification, GIC Architecture Version 3.0 and Version 4.0*. Arm Limited, 2017. 参照第 5 章 Virtual Interrupt Handling and Prioritization.

[7] Arm Limited. *Arm System Memory Management Unit Architecture Specification, SMMU Architecture Version 3.0 and Version 3.1*. Arm Limited, 2017. URL https://documentation-service.arm.com/static/5f900e9ef86e16515cdc0907. 参照第 3.2、3.3、3.6、3.8 节.

[8] Christoffer Dall and Jason Nieh. KVM/ARM: The design and implementation of the linux ARM hypervisor. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 333–348. ACM, 2014. doi: 10.1145/2541940.2541946.

[9] Muli Ben-Yehuda, Michael D. Day, Zvi Dubitzky, Michael Factor, Nadav Har'El, Abel Gordon, Anthony Liguori, Orit Wasserman, and Ben-Ami Yassour. The turtles project: Design and implementation of nested virtualization. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, 2010. URL https://www.usenix.org/conference/osdi10/turtles-project-design-and-implementation-nested-virtualization.

[10] Ori Ben Zur, Jakob Krebs, Shai Aviram Bergman, and Mark Silberstein. Accelerating nested virtualization with HyperTurtle. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 987–1002. USENIX Association, 2025. URL https://www.usenix.org/system/files/atc25-zur.pdf.